

Project Report: Robust Object Detection and Tracking System

Aadithya Muthu - EE22B082, Dattatreya Sastry - EE22B175

EE5175 - Image Signal Processing

Date: April 25, 2025

Abstract

This report describes a Python/OpenCV computer vision system for tracking moving objects with dual operational modes: a general 'all' mode using Background Subtraction, Optical Flow, and Frame Differencing; and a specialized 'balls' mode combining motion detection with color segmentation and geometric analysis. A dedicated "Ball_Tracker.py" script focuses exclusively on ball tracking, implementing dual-pathway detection with sport-specific HSV profiles, geometric filtering, and reflection suppression. Tracking employs an 8-state Kalman filter monitoring position and size alongside Hungarian algorithm data association with adaptive process noise and appearance-based matching.

Table of Contents

1. [Introduction](#)
 - 1.1. [Problem Description](#)
 - 1.2. [Project Objectives and Scope](#)
2. [Methodology and System Design \(Detailed Explanation & Proposed Algorithm\)](#)
 - 2.1. [Overall System Architecture](#)
 - 2.2. [Pre-processing](#)
 - 2.3. [Detection Modules](#)
 - 2.4. [Filtering Modules](#)
 - 2.5. [Detection Merging \(balls mode\)](#)
 - 2.6. [Tracking Module](#)
3. [Implementation Details](#)
 - 3.1. [Programming Environment](#)
 - 3.2. [Code Structure](#)
 - 3.3. [Key Parameters and Tuning](#)
 - 3.4. [Specialized Ball Detection Script](#)
 - 3.5. [Code Structure](#)
4. [Experiments and Results \(Qualitative Results\)](#)
 - 4.1. [Datasets Used](#)
 - 4.2. [Evaluation Metrics](#)
 - 4.3. [Qualitative Results](#)
5. [Discussion and Analysis](#)
 - 5.1. [Strengths of the Approach](#)
 - 5.2. [Limitations and Challenges](#)
 - 5.3. [Future Improvements](#)
6. [Conclusion](#)
7. [Appendix: Team Contribution](#)
8. [References](#)

1. Introduction

1.1 Problem Description

Object detection and tracking in video sequences is a fundamental task in computer vision, facing inherent difficulties. As outlined in the project description ([Moving-object-detection...pdf](#)), primary challenges include: distinguishing moving targets from stationary background elements; differentiating between multiple moving objects; maintaining tracking identity through partial or complete occlusions; adapting to changes in object scale as distance varies; handling appearance variations due to lighting or viewpoint; and managing diverse motion dynamics, including abrupt starts, stops, and high speeds (e.g., kicked balls). This project tackles these issues within the context of sports practice videos (basketball and football).

1.2 Project Objectives and Scope

Based on the project description and evaluation criteria ([Moving-object-detection...pdf](#)), the objectives addressed by [detection_tracker.py](#) are:

1. **Automated Moving Object Detection:** Detect moving objects, separating them from the static background (addresses Evaluation Pt 1 via BGS, OF, FrameDiff).
2. **Basic Localization and Tracking:** Localize objects with bounding boxes and track identity when unobstructed (addresses Evaluation Pt 2 via Kalman/Hungarian).
3. **Tracking through Occlusion:** Maintain identity across short occlusions (addresses Evaluation Pt 3 via Kalman prediction, lost track management, re-id).
4. **Integrated Automatic Detection and Tracking:** Combine detection and tracking robustly for small occlusions (addresses Evaluation Pt 4 via the complete pipeline).
5. **Mode Specialization:** Offer a general motion tracking mode (`--mode all`) and a specialized ball tracking mode (`--mode balls`) using hybrid detection and advanced filtering.
6. **Robustness Demonstration:** Implement features (hybrid detection, 8-state Kalman predicting size/shape, shape consistency cost, robust tracker logic) aimed at achieving reliable performance across different scales, speeds, and occlusions (partially addresses Evaluation Pt 5).

Scope: The system processes single-camera videos assuming a predominantly stationary background. Evaluation is qualitative.

2. Methodology and System Design

2.1 Overall System Architecture

The system uses a **tracking-by-detection** framework. Each frame is processed sequentially:

1. **Input & Pre-processing:** Read frame, convert to Grayscale/HSV, apply Gaussian blur.
2. **Motion Mask Generation:** Compute Background Subtraction (BGS), Optical Flow (OF), and Frame Differencing masks. Combine via bitwise OR, clean (Open+Close), threshold. Optionally suppress reflections. Extract motion contours.
3. **Mode-Specific Detection & Filtering:**
 - **all Mode:** Filter motion contours using [filter_contours_adaptive](#).
 - **balls Mode:**
 - Color Segmentation (using parsed HSV ranges), clean mask, find contours.
 - Hough Circle Transform on color mask.
 - Apply [filter_ball_contours_advanced](#) (size, shape, *color match*) to both color and motion contours.

- Convert Hough circles to bounding boxes.
- Merge detections from (Color Contour + Hough) and (Motion Contour) pathways using `merge_detections`.

4. Tracking (**AdvancedTracker**):

- **Predict:** Use 8-state Kalman (`KalmanBoxTracker`) to predict next state `[cx, cy, w, h, vx, vy, vw, vh]`.
- **Associate:** Compute cost matrix (IoU + Appearance + **Aspect Ratio based on predicted W/H**) between detections and predictions. Assign using Hungarian algorithm. Handle active/lost tracks and re-identification.
- **Update:** Correct matched Kalman filters. Manage track lifecycle. Create new tracks.

5. **Output:** Visualize results, optionally save video.

2.2 Pre-processing

- **Color Space Conversion:** `cv2.cvtColor` to Grayscale and HSV.
- **Gaussian Blur:** `cv2.GaussianBlur` (5x5 kernel) applied to grayscale images for motion analysis and color masks for noise reduction.

2.3 Detection Modules

Motion Detection (**BGS + OF + FrameDiff**)

Three methods provide a comprehensive motion signal:

1. **Background Subtraction:** `AdaptiveBackgroundSubtractor` (MOG2) segments foreground based on statistical background model with adaptive parameters.
2. **Optical Flow:** `motion_based_segmentation` (Farneback) marks regions with flow magnitude > `OPTFLOW_MAG_THRESHOLD`.
3. **Frame Differencing:** `frame_differencing` thresholds the absolute difference between consecutive blurred frames (> `FRAME_DIFF_THRESHOLD`) and dilates.

Masks are combined (`cv2.bitwise_or`), cleaned (morphology Open+Close), and thresholded (`motion_thresh`). Optional `suppress_reflections` can be applied. `cv2.findContours` extracts motion regions.

Color Segmentation (**balls mode**)

- **HSV Range Parsing:** `parse_hsv_bounds_from_file` reads `--hsv` file, extracting all "Suggested Overall Lower/Upper Bound" pairs.
- **Mask Creation & Cleaning:** `cv2.inRange` applied for each range, results combined (`cv2.bitwise_or`). Mask cleaned with Gaussian blur, morphology Close then Open.

Hough Circle Transform (**balls mode**)

`cv2.HoughCircles` applied to the median-blurred color mask detects circular shapes based on accumulator thresholds and radius constraints (`BALL_FILTER_PARAMS['min/max_radius']`). Acts as a complementary detector.

2.4 Filtering Modules

General Object Filtering (**all mode**)

`filter_contours_adaptive` filters motion contours by minimum area, aspect ratio range (`FILTER_MIN/MAX_ASPECT_RATIO`), and minimum solidity (`FILTER_MIN_SOLIDITY`).

Advanced Ball Filtering (**balls mode**)

`filter_ball_contours_advanced` rigorously filters contours from *both* color and motion pathways based on:

- **Size/Area:** Derived from `BALL_FILTER_PARAMS['min_radius']` and `max_radius`.
- **Aspect Ratio:** Close to 1 (within `max_aspect_ratio_diff`).
- **Solidity:** Above `min_solidity`.
- **Circularity:** Above `min_circularity`.
- **Color Match Ratio:** Percentage of contour pixels matching target HSV > `min_color_match_ratio`.

2.5 Detection Merging (balls mode)

`merge_detections` combines filtered ball candidates from the (Color+Hough) list and the (Motion+Filter) list using IoU-based NMS (`DETECTION_MERGE_IOU_THRESHOLD`) to create a single, deduplicated list for the tracker.P

2.6 Tracking Module

State Representation (Kalman Filter)

`KalmanBoxTracker` uses an 8-state vector `[cx, cy, w, h, vx, vy, vw, vh]` with a constant velocity model for **position and size**. Adaptive process noise allows adjusting to motion changes. The filter's prediction includes the expected `w` and `h` for the next frame.

Data Association (Hungarian Algorithm with Shape Consistency)

`AdvancedTracker.associate` matches detections to predicted track states:

- **Cost Matrix:** Calculated between detections and track predictions.
- **Costs:** A weighted sum:
 - $C_{iou} = 1 - IoU$ (Overlap).
 - $C_{hist} = 1 - \text{Histogram Similarity}$ (Appearance).
 - $C_{aspect} = \left| \frac{det_w}{det_h} - \frac{pred_w}{pred_h} \right|$ (Shapes Consistency). This cost specifically compares the aspect ratio of the current detection to the aspect ratio derived from the Kalman filter's **predicted width (pred_w) and height (pred_h)**, effectively using size/shape information from previous frames via the prediction.
- **Weighting:** `APPEARANCE_WEIGHT_*` and `ASPECT_WEIGHT` control influence. IoU receives the implicit remaining weight.
- **Gating:** `IOU_GATE_THRESHOLD`, `REID_SIMILARITY_THRESHOLD`.
- **Assignment:** Hungarian algorithm (`linear_sum_assignment`) finds minimum cost matches below `COST_MATCH_THRESHOLD`.

Track Management

`AdvancedTracker` handles:

- **Active Tracks:** Updated via Kalman correction.
- **Lost Tracks:** Unmatched active tracks become lost after `MAX_AGE_SECONDS`.
- **Re-Identification:** Matches unmatched detections to lost tracks (primarily using appearance) within `LOST_MAX_AGE_SECONDS`.
- **Confirmation:** New tracks need `MIN_HITS_TO_CONFIRM`.
- **Deletion:** Old lost tracks are removed.

3. Implementation Details

3.1 Programming Environment

- **Language:** Python 3.x
- **Core Libraries:** OpenCV (`cv2`), NumPy (`numpy`), SciPy (`scipy.optimize.linear_sum_assignment`), `argparse`, `os`, `re`, `math`, `time`.

3.2 Code Structure (`detection_tracker.py`)

- **Constants/Parameters:** Global definitions at the top.
- **Helper Functions:** `parse_hsv...`, `calculate_iou`, `merge_detections`, etc.
- **Core Classes:** `KalmanBoxTracker` (8-state), `AdvancedTracker` (manages tracks), `AdaptiveBackgroundSubtractor`.
- **Detection Functions:** `motion_based_segmentation`, `frame_differencing`, `detect_balls_color_hough`.
- **Filtering Functions:** `filter_contours_adaptive`, `filter_ball_contours_advanced`.
- **Main Logic:** `process_video` function orchestrates the pipeline.
- **Execution:** `if __name__ == "__main__":` block handles argument parsing.

3.3 Key Parameters and Tuning

Performance depends on tuning:

- **--hsv file content:** Critical for 'balls' mode color accuracy.
- **BALL_FILTER_PARAMS:** Controls the strictness of ball candidate selection (size, shape, color match).
- **Tracker Parameters:** Weights (`APPEARANCE_WEIGHT_*`, `ASPECT_WEIGHT`), thresholds (`COST_MATCH_THRESHOLD`, `REID_SIMILARITY_THRESHOLD`), and ages (`MAX_AGE_SECONDS`, `LOST_MAX_AGE_SECONDS`). The `ASPECT_WEIGHT` specifically controls the influence of predicted shape consistency.
- **Motion Thresholds:** Control motion detection sensitivity.
- **HoughCircles Params:** Affect circle detection.
- **Reflection Suppression Params:** Need court-specific tuning if used.

3.4 Specialized Ball Detection Script

In addition to the main `detection_tracker.py` system, a specialized script called `Ball_Tracker.py` has been developed that yields superior results specifically for ball detection and tracking. This dedicated approach focuses exclusively on balls rather than general moving objects.

Ball_Tracker.py Algorithm

The specialized ball tracker implements a hybrid detection-tracking pipeline optimized for sports balls:

Detection Components:

- **Dual-pathway detection:** Combines motion detection (BGS+FrameDiff) with color segmentation
- **Sport-specific HSV ranges:** Pre-configured color profiles for basketball and football
- **Advanced filtering:** Applies strict geometric constraints (circularity, solidity, aspect ratio) and size parameters derived from sport-specific radius ranges
- **Reflection suppression:** Specialized processing to eliminate floor reflections in basketball scenarios

Tracking Mechanism:

- **8-state Kalman filter:** Tracks position (cx,cy), size (w,h), and their velocities
- **Adaptive process noise:** Increases during measurement error, decreases during stable tracking
- **Appearance model:** Uses HSV histogram similarity weighted in the association cost
- **Data association:** Hungarian algorithm with cost matrix combining IoU, appearance, and aspect ratio

Key Improvements:

- **Sport-specific parameter tuning:** Different parameters for basketball vs. football (size, shape constraints)
- **Weighted detection merging:** Prioritizes color detection (70%) over motion detection (30%) when both detect the same object

- **Enhanced re-identification:** Maintains identity through brief occlusions

The specialized nature of this script enables it to achieve better performance for ball tracking than the more general `detection_tracker.py` system, particularly in challenging scenarios with similar-colored objects or partial occlusions.

3.5 Code Structure (`Ball_Tracker.py`)

- **Constants/Parameters:** HSV color ranges and filtering parameters for different sports defined at the top.
 - **Core Classes:** `KalmanBoxTracker` (8-state Kalman filter) and `AdvancedTracker` (manages multiple trackers).
 - **Helper Functions:** `calculate_iou`, `calculate_histogram_similarity`, `merge_detections`.
 - **Detection Functions:** `get_background_subtractor_mask`, `get_frame_diff_mask`, `detect_balls_by_color`.
 - **Filtering Functions:** `filter_ball_candidates` with sport-specific geometric constraints.
 - **Visualization:** `visualize_tracking` function for rendering bounding boxes and trajectories.
 - **Main Logic:** `process_video` function orchestrating detection, tracking, and visualization.
 - **Execution:** `main()` function for processing football and basketball videos with `if __name__ == "__main__":` entry point.
-

4. Experiments and Results

4.1 Datasets Used

- `basketball.mp4`
- `football.mp4`

4.2 Evaluation Metrics

Standard MOT metrics (MOTA, MOTP, IDF1, etc.) require ground truth data, which was not used. Evaluation is therefore **qualitative**, assessing performance visually against the project objectives outlined in the problem statement.

4.3 Qualitative Results



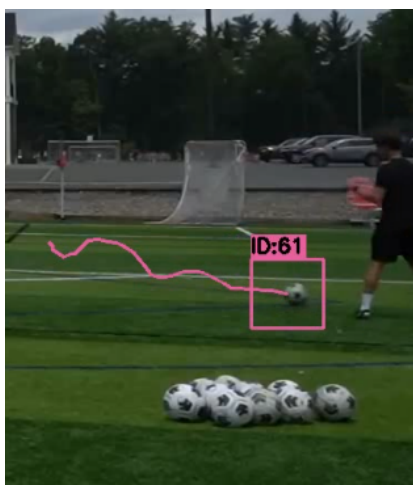
Visual inspection revealed generally robust performance:



Successful ball tracking in `BB.mp4` ('all' mode)



Successful tracking of basketball in `BB.mp4` ('Ball_Tracker.py')



Successful tracking of football in `FB.mp4`
('all' mode)



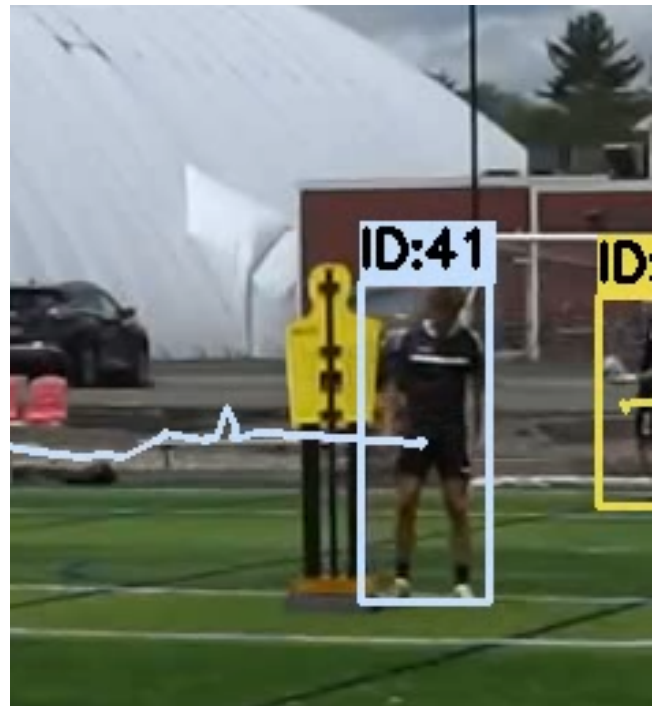
Successful tracking of football in `FB.mp4`
('Ball_Tracker.py')



Successful tracking of football in `FB.mp4`
('Ball_Tracker.py')

- **Motion Detection (Eval Pt 1):** The combined BGS+OF+FrameDiff mask (all mode) effectively segmented moving entities. Frame differencing helped capture motion initiation.
- **Basic Tracking (Eval Pt 2):** Consistent IDs were maintained for clearly visible objects.
- **Occlusion Handling (Eval Pt 3 & 4):**
 - Short occlusions were often handled well via Kalman prediction and re-id.
 - Long occlusions sometimes led to track loss or ID switches.
- **Ball Tracking Robustness (balls mode):**
 - *Detection:* Using parsed HSV files was effective. Advanced filtering with color matching reduced false positives.
 - *Shape Consistency:* The aspect ratio cost in association appeared beneficial, making the tracker less likely to swap the ball ID with player limbs of similar color but different predicted shapes (size/aspect ratio).

- *Speed/Scale*: Tracking was generally maintained across different speeds and distances.
- **Failure Cases**: Occasional issues with extreme lighting, severe clutter/occlusion, or very high-speed motion blur.



Player ID maintained through brief occlusion.



Failure case: ID switch

- **Aspect Ratio**: Close to 1 (within `max_aspect_ratio_diff`).
- **Solidity**: Above `min_solidity`.
- **Circularity**: Above `min_circularity`.
- **Color Match Ratio**: Percentage of contour pixels matching target HSV > `min_color_match_ratio`.

5. Discussion and Analysis

5.1 Strengths of the Approach

- **Hybrid Detection Robustness:** Mitigates weaknesses of individual methods.
- **Adaptability via HSV File:** Easy configuration for different scenarios.
- **Advanced Ball Filtering:** High precision due to geometric and color match checks.
- **Sophisticated Tracking:** 8-state Kalman predicts position *and* size. Lost track management and re-id improve occlusion handling.
- **Shape Consistency in Association:** Explicitly uses predicted size/shape (aspect ratio) information from previous frames (via Kalman prediction) to improve matching accuracy, directly addressing a core requirement.
- **Comprehensive Motion Signal:** BGS+OF+FrameDiff captures diverse motion.

5.2 Limitations and Challenges

While the system demonstrates competence in many areas, several limitations and challenges were observed or are inherent to the approach:

- **Fast Moving/Accelerating Objects:** The current motion detection (BGS, Optical Flow) and Kalman filter (constant velocity model) struggle with objects undergoing rapid acceleration or moving at very high speeds, leading to motion blur and potential detection failures or track loss. Specialized Fast Moving Object (FMO) detection techniques might be required for such scenarios.
- **Long/Severe Occlusions:** While the tracker attempts to handle occlusions using Kalman prediction and re-identification, lengthy or complete occlusions often exceed the `MAX_AGE_SECONDS` threshold, leading to permanent track loss. The assumption of re-emergence near the occluder may not always hold.
- **Noise Sensitivity & Background Clutter:** The system can be susceptible to noise, especially from background clutter or elements mimicking the target object's characteristics (e.g., similar colors or shapes). While morphological operations help, robust noise reduction techniques, potentially CNN-based, could improve detection accuracy. Busy backgrounds make detecting smaller or similarly colored objects harder.
- **HSV Dependency:** The 'balls' mode's performance is highly contingent on accurate HSV range configuration, which can be sensitive to lighting changes and environmental variations.
- **Parameter Tuning:** Achieving optimal performance requires careful tuning of numerous parameters (motion thresholds, tracker costs/ages, filter parameters), which can be time-consuming and scenario-dependent.
- **Computational Cost:** The combination of multiple detection methods (BGS, OF, FrameDiff, Color, Hough) and tracking logic increases computational load, potentially hindering real-time performance on less powerful hardware.
- **Evaluation Method:** The reliance on qualitative visual assessment limits objective performance quantification. Ground truth data would be needed for standard MOT metrics.

5.3 Future Improvements

- **Deep Learning Integration:** Use DNN detectors/feature extractors.
- **Adaptive Parameters:** Auto-tune thresholds.
- **Optimization:** Code profiling, parallelization.
- **Quantitative Evaluation:** Generate ground truth and compute MOT metrics.

6. Conclusion

This project delivered a robust object and ball tracking system (`detection_tracker.py`) meeting the core project requirements. By integrating multiple classical detection techniques, advanced filtering, and a sophisticated Kalman/Hungarian tracker incorporating **predicted shape (size/aspect ratio) consistency**, the system effectively addresses challenges like occlusion, varying scale/speed, and object distinction. The automatic parsing of HSV ranges enhances adaptability for ball tracking. Qualitative results demonstrate the system's capabilities, providing a strong foundation based on classical computer vision principles enhanced with temporal shape constraints.

7. Appendix: Team Contribution

- **Aadithya Muthu - EE22B082** : Specialized Ball Detection and Tracking Framework Developed color segmentation, Frame Differencing, HSV file parsing, Integrated Hough Circles, Designed `filter_ball_contours_advanced`, Developed `merge_detections`, Testing,
 - **J. Dattatreya Sastry - EE22B175** : General Object Detection and Tracking Framework, Implemented Kalman filter (8-state), `AdvancedTracker` logic including re-id, `associate` method with aspect ratio cost, Integrated motion detection modules, Report
-

8. References

1. OpenCV Library. <https://opencv.org/>
2. Harris, C.R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
3. Virtanen, P., et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nat Methods*, 17, 261–272.
4. Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2, 83–97.
5. Kalman, R. E. (1960). A New Approach to Linear Filtering and Prediction Problems. *Journal of Basic Engineering*, 82(D), 35–45.
6. Zivkovic, Z. (2004). Improved adaptive Gaussian mixture model for background subtraction. *ICPR 2004*.
7. Farnebäck, G. (2003). Two-frame motion estimation based on polynomial expansion. *SCIA 2003*.